VISUAL RECOGNITION FINAL PROJECT

This project is worth **15 marks** and requires students to independently design and implement a complete visual recognition pipeline using a self-selected real-world image dataset.

The final system must be demonstrably usable and applicable to a real-world scenario.

DATASET SELECTION

Each team must select an image dataset that satisfies the following criteria:

- Minimum 500 labeled images across at least 3 distinct classes
- Publicly available from a reputable source such as Roboflow Universe, Kaggle, Open Images, or a custom-collected dataset
- A unique and creative dataset that is relevant to a real-world domain and provides practical value
- Images must have sufficient variation in lighting, background, scale, and angle to make augmentation meaningful

PROJECT TASKS

Task 1: Dataset Acquisition & Exploration

Collect or download your dataset and conduct an initial exploratory analysis. Your notebook must include:

- Total image count, class distribution, and image resolution statistics
- A minimum of three visualizations: class balance chart, sample grid of images per class, and image size/resolution distribution
- A discussion of class imbalance, quality issues, or data challenges observed
- Dataset source, justification for its selection, and real-world relevance

Task 2: Data Preprocessing & Augmentation

Prepare and expand the dataset to improve model generalization. You must apply multiple augmentation techniques (if needed) and justify each one based on your domain. Examples include:

- Geometric transforms: flipping, rotation, cropping, perspective warping, shearing
- Photometric transforms: grayscale conversion, channel dropout, histogram equalization... etc.

Provide before-and-after visual comparisons for each technique applied. Clearly state how augmentation affects the effective dataset size.

Task 3: YOLO Model Training

Train a YOLO-based object detection model using your processed dataset. Your implementation must include:

- Dataset splitting into training, validation, and test sets with justification
- Performance evaluation, including: Confusion matrix, Per-class performance, and Map score
- Visual results: detection outputs on test images with bounding boxes and confidence scores

- **Document any hyperparameter tuning** (e.g., learning rate, batch size, augmentation probability).
- **Compare at least two configurations** and justify your final model choice.
- **Evaluate and compare your trained model with one pretrained model** (e.g., YOLOv5, YOLOv8) or any other relevant pretrained detection model suitable for your dataset

Task 4: Real-Time Detection System

Develop a real-time detection application using your trained model. The system must:

- Accept input from a webcam or video file using OpenCV
- Perform inference on each frame and overlay bounding boxes, class labels, and confidence scores
- Display FPS (frames per second) and maintain real-time performance
- Include a user-controllable confidence threshold (via a slider or keyboard input)
- **1 bonus mark:** trigger a domain-specific action based on detections (e.g., log an alert, play a sound, highlight a region)

The system must be fully runnable and demonstrated. You should test and report its average inference speed (FPS) and discuss any limitations encountered in real-time conditions.

Deliverables

- A fully executed **Python Notebook** with all outputs visible
- Clear explanations using markdown cells
- Demonstration of all required tasks

Good luck!